

ENTREGA 2 – OSHI NO BATTLE RPG

1. DESCRIPCIÓN GENERAL

El sistema implementa un videojuego RPG por turnos completamente funcional, incluyendo:

- *Motor de combate*
- *Gestión de personajes y enemigos*
- *Sistema de habilidades*
- *Inventario*
- *Progresión (nivel y experiencia)*
- *Persistencia de partida*
- *Interfaz gráfica interactiva*

2. MOTOR DE COMBATE

*El combate se desarrolla por turnos utilizando la clase **BattleSystem**.*

Flujo de combate

1. *Se inicializa el orden de turnos (**TurnManager**)*
2. *Se selecciona el personaje actual*
3. *Se ejecuta acción:*
 - *Jugador → input manual*
 - *Enemigo → IA automática*
4. *Se actualiza estado del juego*

5. Se repite hasta condición de victoria o derrota

Acciones implementadas

- **attack()** → *daño directo considerando defensa*
- **skill()** → *habilidades con consumo de mana*
- **defend()** → *reduce daño recibido*
- **useItem()** → *aplica efectos del inventario*

IA enemiga

- *Selecciona objetivo aleatorio vivo*
- *Decide entre:*
 - *Usar skill (si tiene mana)*
 - *Atacar*

3. USO DE COLECCIONES

Se utilizan estructuras de datos dinámicas:

Listas (ArrayList)

- **GameState.party**
- **GameState.enemies**
- **Inventory.items**
- **GameCharacter.skills**

Cola (Queue)

- *TurnManager.queue*

Permite manejar turnos de forma ordenada y dinámica.

4. LÓGICA DE NEGOCIO

Sistema de daño

```
int finalDamage = Math.max(0, dmg - defense);  
hp -= finalDamage;
```

Se tiene en cuenta la defensa

No permite daño negativo

Estados

- *DEFEND* → reduce daño recibido
- Sistema preparado para expandirse (*POISON, STUN*)

Progresión

- Ganancia de experiencia al ganar batalla
- Subida de nivel automática:

```
if(exp >= level*100){  
    levelUp();  
}
```

Mejora atributos del personaje

5. INVENTARIO

Clase *Inventory* permite:

- *Agregar ítems*
- *Usar ítems sobre personajes*

Tipos de ítems:

- *heal* → *recupera HP*
- *mana* → *recupera MP*
- *buff* → *aumenta ataque*

6. PERSISTENCIA

Se implementa guardado y carga usando serialización:

ObjectOutputStream

ObjectInputStream

Se guarda:

- *Party*
- *Enemigos*
- *Inventario*

Características:

- *Archivo local (save.dat)*
- *Recuperación automática al iniciar*

7. MANEJO DE EXCEPCIONES

Se utilizan bloques **try-catch** para evitar fallos:

```
try(ObjectOutputStream out = new ObjectOutputStream(...)){  
    ...  
}catch(Exception e){  
    ...  
}
```

Evita crash si no existe archivo Permite iniciar nueva partida automáticamente

8. INTERFAZ GRÁFICA (SWING)

Pantallas implementadas

Inicio

- ***Nueva partida***
- ***Cargar partida***

Selección

- ***Personajes con imagen***
- ***Selección aliados/enemigos***

Batalla

- ***Visualización de stats***
- ***Selección de enemigo***
- ***Acciones disponibles***

Resultado

- *Ganar / perder*
- *Tiempo de partida*

Características UI

- *Imágenes de personajes*
- *Fondo visual*
- *Bordes para selección*
- *Log de acciones en tiempo real*

9. INTEGRACIÓN DEL SISTEMA

El sistema funciona de forma integrada:

- *GameUI* → *interacción usuario*
- *BattleSystem* → *lógica del combate*
- *GameState* → *estado global*
- *TurnManager* → *orden de turnos*

Separación clara de responsabilidades y Flujo coherente entre capas

10. DECISIONES DE DISEÑO

Uso de clases separadas

Permite modularidad y escalabilidad.

Uso de enums

Simplifica lógica de roles y estados.

Serialización simple

Elegida por simplicidad y rapidez de implementación.

IA básica

Suficiente para cumplir requisitos sin complejidad excesiva.

CONCLUSIÓN

El sistema cumple con todos los requerimientos de la fase:

Motor de combate funcional

Uso de colecciones

Lógica completa

Manejo de errores

Persistencia

Interfaz gráfica operativa

- *Arquitectura: “Separé lógica, estado y UI para mantener bajo acoplamiento.”*
- *Persistencia: “Utilicé serialización para guardar el estado completo del juego.”*
- *Turnos: “El sistema usa una cola ordenada por velocidad para gestionar turnos.”*
- *Escalabilidad: “El diseño permite agregar nuevos personajes, skills o estados fácilmente.”*

ENTREGA FINAL – DEFENSA INDIVIDUAL OSHI NO BATTLE RPG

1.Explicación del diseño individual

Arquitectura general

- **GameUI** → Interfaz gráfica y flujo de interacción del usuario.
- **BattleSystem** → Motor de combate: turnos, habilidades, acciones de jugador y IA enemiga.
- **GameState** → Estado global del juego, incluyendo personajes, enemigos e inventario.
- **TurnManager** → Cola de turnos basada en velocidad.
- **GameCharacter, Skill, Item, Inventory** → Modelado de entidades principales.

Principios aplicados

- **Modularidad** → Separación clara entre UI, lógica de negocio y estado.
- **Reutilización** → Clases **GameCharacter** y **Skill** son genéricas, permiten añadir nuevos personajes o habilidades.
- **Escalabilidad** → Nuevos estados, skills o items se pueden agregar sin modificar la lógica central.
- **Persistencia** → Serialización simple para guardar y cargar partidas.

2.Resolución de un ejercicio breve

Ejercicio: Modificar el código para que los enemigos también puedan usar items del inventario si su HP es bajo.

Solución propuesta (ejemplo):

```
private static void enemyAI(GameCharacter enemy){  
    List<GameCharacter> vivos = GameState.party.stream()
```

```

        .filter(GameCharacter::isAlive)
        .toList();

if(vivos.isEmpty()) return;

GameCharacter target = vivos.get(new Random().nextInt(vivos.size()));

// Nuevo comportamiento: enemigo usa item si HP < 30%
if(enemy.hp < enemy.maxHp*0.3 && !GameState.inventory.items.isEmpty()){
    GameState.inventory.use(0, enemy);
    ui.log(enemy.name + " usó un ítem para curarse!");
}
else if(enemy.mana >= 10 && !enemy.skills.isEmpty()){
    Skill s = enemy.skills.get(0);
    s.use(enemy, target);
    ui.log(enemy.name + " usó skill " + s.name + " sobre " + target.name);
} else {
    target.takeDamage(enemy.attack);
    ui.log(enemy.name + " atacó a " + target.name + " (-" + enemy.attack + " HP)");
}

endTurn();
}

```

Justificación:

- *Mantiene coherencia con la lógica existente.*
- *Demuestra que las colecciones (**Inventory.items**) y métodos (**use()**) se integran de forma genérica.*
- *Permite extensión futura sin modificar la base de combate.*

3.Justificación de decisiones de diseño

Área	Decisión	Justificación
------	----------	---------------

Separación de capas	UI / Lógica / Estado	Facilita mantenimiento y pruebas unitarias
Serialización	Guardado en archivo <i>save.dat</i>	Simplicidad y rapidez de implementación
Cola de turnos	<i>Queue</i> + orden por velocidad	Mantiene turnos dinámicos y justos, escalable
Estados de personaje	<i>StatusEffect</i> (DEFEND, POISON...)	Flexible para implementar efectos y habilidades
Habilidades	Clase <i>Skill</i> genérica	Permite expandir con nuevos tipos (damage, heal, buff)
Imágenes y UI	<i>ImageIcon</i> por personaje	Mejor experiencia visual y diferenciación de personajes

4.Integración completa del proyecto

- **Todas las piezas del sistema están conectadas:**
 - **UI permite seleccionar personajes/enemigos y ejecutar combate**
 - **BattleSystem interactúa con GameState y TurnManager**
 - **Inventario y Skills afectan directamente los personajes**
- **Log de acciones muestra cambios de HP/MP en tiempo real para jugadores y enemigos**
- **Persistencia permite cerrar y continuar la partida sin pérdida de datos**
- **Interfaz gráfica funcional con imágenes, bordes y fondos diferenciados para cada pantalla**

5.Preguntas y respuestas posibles para la defensa

Pregunta: ¿Cómo manejarías nuevos tipos de habilidades?

Respuesta:

- Agregando un nuevo **type** en la clase **Skill** y extendiendo el método **use()**.
- No se modifica la lógica central, solo la acción específica del skill.

Pregunta: ¿Qué pasa si el archivo de save se corrompe?

Respuesta:

- El bloque **try-catch** captura errores y crea una partida nueva automáticamente.

Pregunta: ¿Cómo aseguraste que los enemigos también pierdan HP visualmente?

Respuesta:

- En **BattleSystem.attack()** y **BattleSystem.skill()**, se refresca la UI después de cada acción para reflejar cambios de HP de jugadores y enemigos.

Pregunta: ¿Cómo agregarías más personajes?

Respuesta:

- Se agregan en **GameState.getAllCharacters()** con imagen y habilidades.
- La selección y el combate funcionan automáticamente gracias a la integración modular.

6. Resumen para la defensa oral

- Explicar la arquitectura modular
- Mostrar la interfaz y combate funcionando
- Justificar decisiones sobre colecciones, persistencia y lógica
- Mostrar un ejercicio de modificación simple (como el ejemplo de items enemigos)
- Responder preguntas destacando la escalabilidad y reutilización